

LC 507: Perfect Number

Authored by: Arpit Raj, LNMIIT Jaipur

October 1, 2026

Core Concept

A **Perfect Number** is a positive integer that is equal to the sum of its positive divisors, excluding the number itself. We need to find all proper divisors, sum them up, and check if the sum equals `num`.

Edge Case: If `num == 1`, it has no proper divisors (excluding itself). Therefore, we immediately return `false`;

Optimized Divisor Summation

Initial State

Since 1 is always a proper divisor for any `num > 1`, we can initialize our sum to 1.

```
int sum = 1;
```

Loop Condition

Instead of looping all the way to `num`, we only loop up to the square root: `d * d <= num`. This reduces time complexity to $O(\sqrt{N})$.

Handling Paired Divisors

When we find a divisor `d` (i.e., `num % d == 0`), we add it to our sum. But divisors come in pairs! The paired divisor is `num / d`.

```
sum += d;
if (d * d != num) {
    sum += num / d; // Add the pair if it's not a perfect square
}
```

Note: We check `d * d != num` to ensure we don't add the square root twice (e.g., for 36, we only add 6 once).

C++ Implementation

Full code implementing the optimized $O(\sqrt{N})$ divisor logic.

```
class Solution {
public:
    bool checkPerfectNumber(int num) {

        if(num==1) return false;
        int sum=1;
        for(int d=2;d*d<=num;d++){
            if(num%d==0){
                sum+=d;
                if(d*d!=num){
                    sum+=num/d;
                }
            }
        }

        if(sum==num) return true;
        return false;
    }
};
```